



ConfidenceOS

The HMI That Knows What It Does Not Know

FINAL PROJECT DOCUMENTATION

Next-Generation Control System Interface

Read-only trust-aware HMI overlay, verification workflow, handover system, and model-driven Runtime generation platform.

Project Snapshot

Read-only No process control writes	1 Hz Live runtime stream	5 roles Operator to auditor
4-factor Confidence rubric	Studio Generated Runtime flow	Audit Ledger and evidence trail
Core idea Traditional HMIs show values. ConfidenceOS shows values together with whether those values deserve trust, what evidence supports them, what decisions should be frozen, and what must be handed over.		

1. Executive Summary

ConfidenceOS is a read-only, trust-aware Human-Machine Interface layer for industrial operations. It is designed for situations where operators cannot safely rely only on raw sensor values because a displayed value may be stale, miscalibrated, physically implausible, or contradicted by neighboring process data.

The project adds a trust layer beside the HMI/DCS. It reads live or replayed tag values, scores whether readings deserve trust, checks physical consistency using a mass-balance residual, identifies abnormal situations, creates field-verification tasks, preserves handover debt, and generates operator Runtime screens through an engineering Studio.

The outcome is an operator interface that does not simply display data. It explains which data can be used as operating basis, which data should not be trusted, what evidence replaces it, and what action must happen next.

Judging message

ConfidenceOS is not an alarm filter. It is a trust-aware operating layer that addresses the deeper problem of false certainty in industrial interfaces.

2. Problem Statement

Industrial operators make high-consequence decisions from HMI screens. However, most HMIs display values with equal visual confidence. A transmitter can drift, freeze, or be contradicted by process physics while the screen still presents the reading as a normal number.

This creates a dangerous gap between **what the system displays** and **what is physically true**. ConfidenceOS is built to close that gap.

Traditional HMI:

- Shows the sensor value.

ConfidenceOS:

- Shows the sensor value.
- Shows whether the value should be trusted.
- Shows why.
- Shows what decision should be frozen.
- Shows what verification is required.
- Preserves the issue for shift handover.

Incident Motivation

The project is motivated by industrial events where operators had data but not enough trust context. A level reading may look safe while mass balance says the vessel inventory is rising. A valve may show commanded state instead of actual state. A temperature may be displayed without enough context about whether that state is physically abnormal for the equipment.

ConfidenceOS turns those hidden uncertainty conditions into visible operator information.

3. Project Objectives

- Attach a clear trust state and confidence score to live sensor readings.

- Use calibration, stability, cross-sensor consistency, and physical plausibility as evidence.
- Detect physical contradictions using mass-balance residual checking.
- Translate technical evidence into operator-facing guidance.
- Generate field-verification tasks when trust cannot be established from screen data alone.
- Preserve unresolved operating debt across shift handover.
- Generate Runtime HMI screens from metadata, asset models, templates, and approved mappings.
- Keep the system read-only and separate from plant control authority.

4. Proposed Solution

ConfidenceOS sits beside existing control-system infrastructure as a read-only decision-support layer. It does not replace the DCS, SIS, historian, CMMS, or operator authority. It observes data, evaluates trust, and presents evidence.

Runtime

Operator-facing HMI screen that shows live values, trust state, abnormal situation, decision freeze, trusted substitutes, and single safe move.

Studio

Engineering workspace for tag import, Mapping Court, template assignment, validation, build receipts, and Runtime publication.

Work Queue

Maintenance and engineering workflow for field verification with structured evidence and lifecycle state.

Shift Channel

Handover console that preserves unresolved trust debt, verification tasks, notes, and operational trace references.

5. Feature Overview

Capability	What it delivers to the project
Trust-aware Runtime HMI	Every value is presented with a trust state, confidence evidence, and operating guidance.
Confidence Scoring Engine	Combines calibration age, stability, cross-sensor evidence, and physical plausibility into a transparent trust rubric.
Mass-Balance Residual Check	Uses conservation-of-volume logic to detect physically suspicious disagreement between level and flows.
Trust Quarantine	Prevents a contradicted sensor value from being treated as a valid decision basis.
Field Verification Workflow	Turns low-trust readings into actionable maintenance and engineering verification tasks.
Shift Channel	Preserves unresolved trust debt, verification state, notes, and operational context across shift handover.
Studio and HMI Compiler	Supports tag import, Mapping Court, template assignment, validation, build receipts, and published Runtime manifests.
Operational Ledger	Creates an auditable trail of events, notes, verification transitions, and handover-critical information.
Role-aware Workspaces	Gives operators, maintenance, engineers, managers, and auditors focused views of the same system state.

6. System Architecture

The architecture separates read-only data sources, runtime intelligence, engineering Studio, persistence, and frontend workspaces. This separation keeps the safety boundary clear while allowing the Runtime display to be generated from engineering metadata.

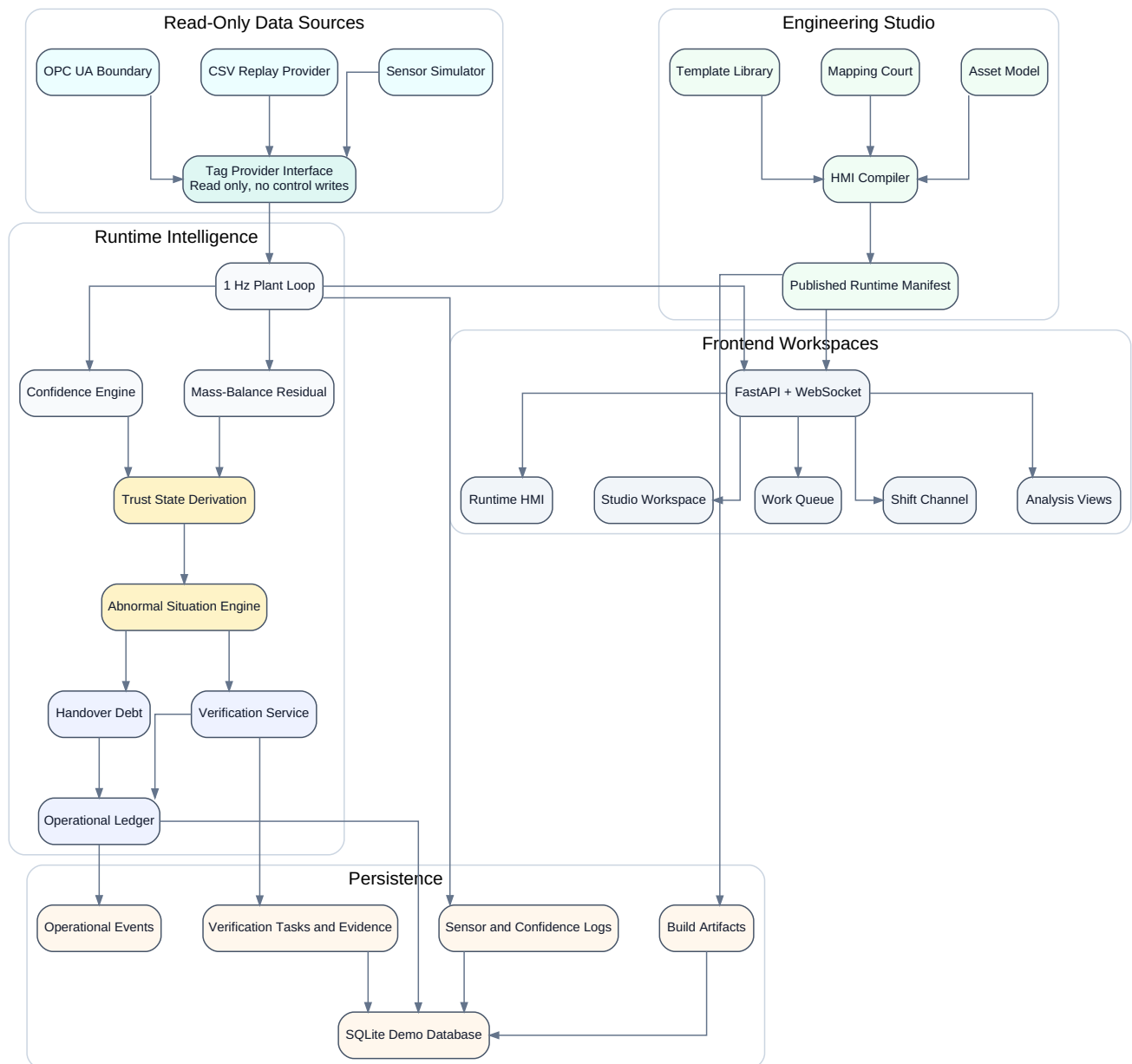


Figure 6.1 shows the ConfidenceOS architecture across read-only data sources, runtime intelligence, engineering Studio, persistence, and frontend workspaces.

7. Runtime Operating Flow

Runtime processing begins with live tag readings and ends with operator-facing decision support. The main value is not the confidence score alone, but the conversion of that score into trust state, operating basis, action, verification, and handover continuity.

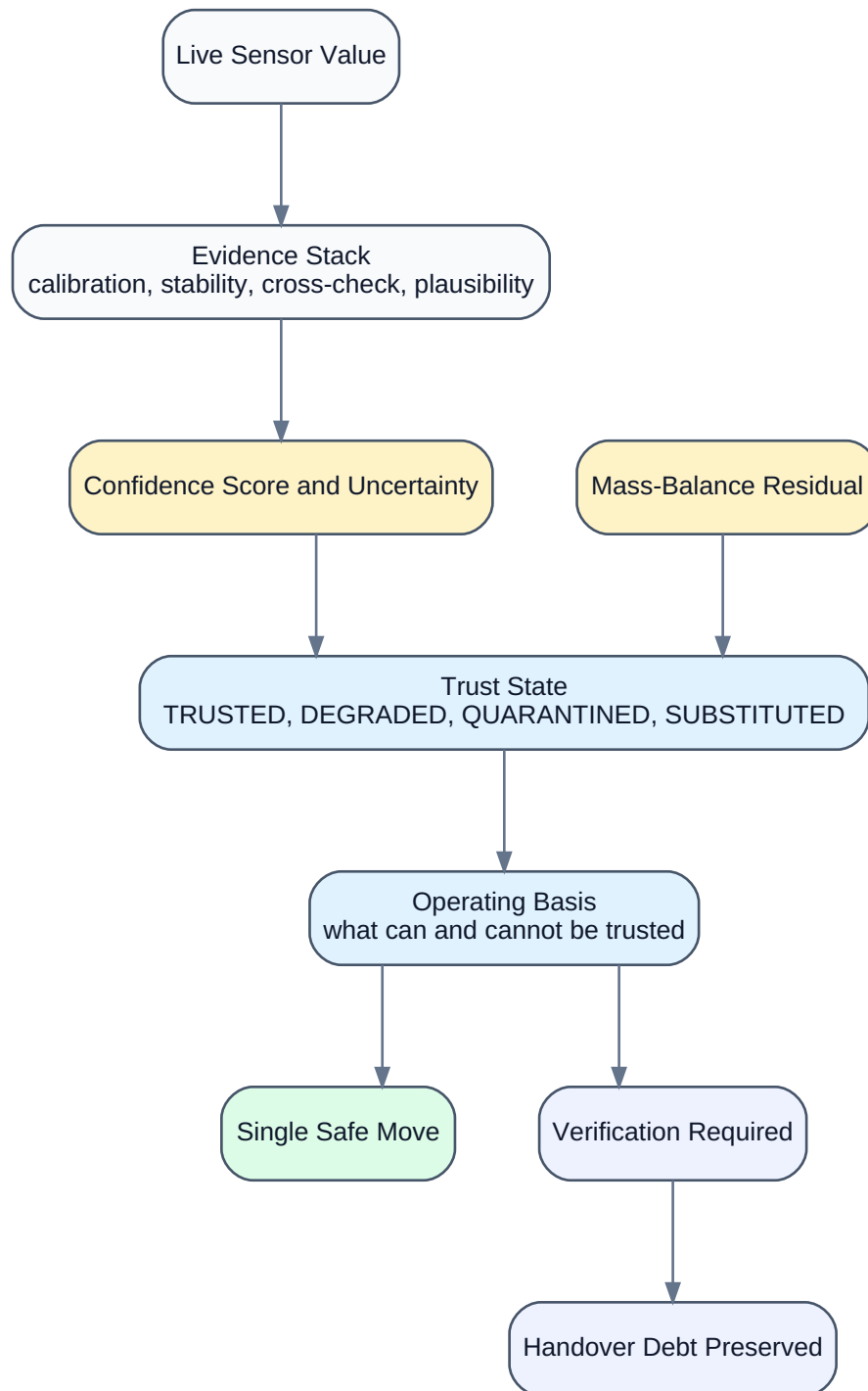


Figure 7.1 shows how ConfidenceOS converts live sensor values into trust state, operating basis, safe action, verification, and handover debt.

8. HMI Compiler and Studio Flow

Studio is the engineering side of ConfidenceOS. It helps convert raw tags and asset metadata into usable Runtime screens. The compiler flow ensures that unresolved or ambiguous tags are not silently published as operator-facing screens.

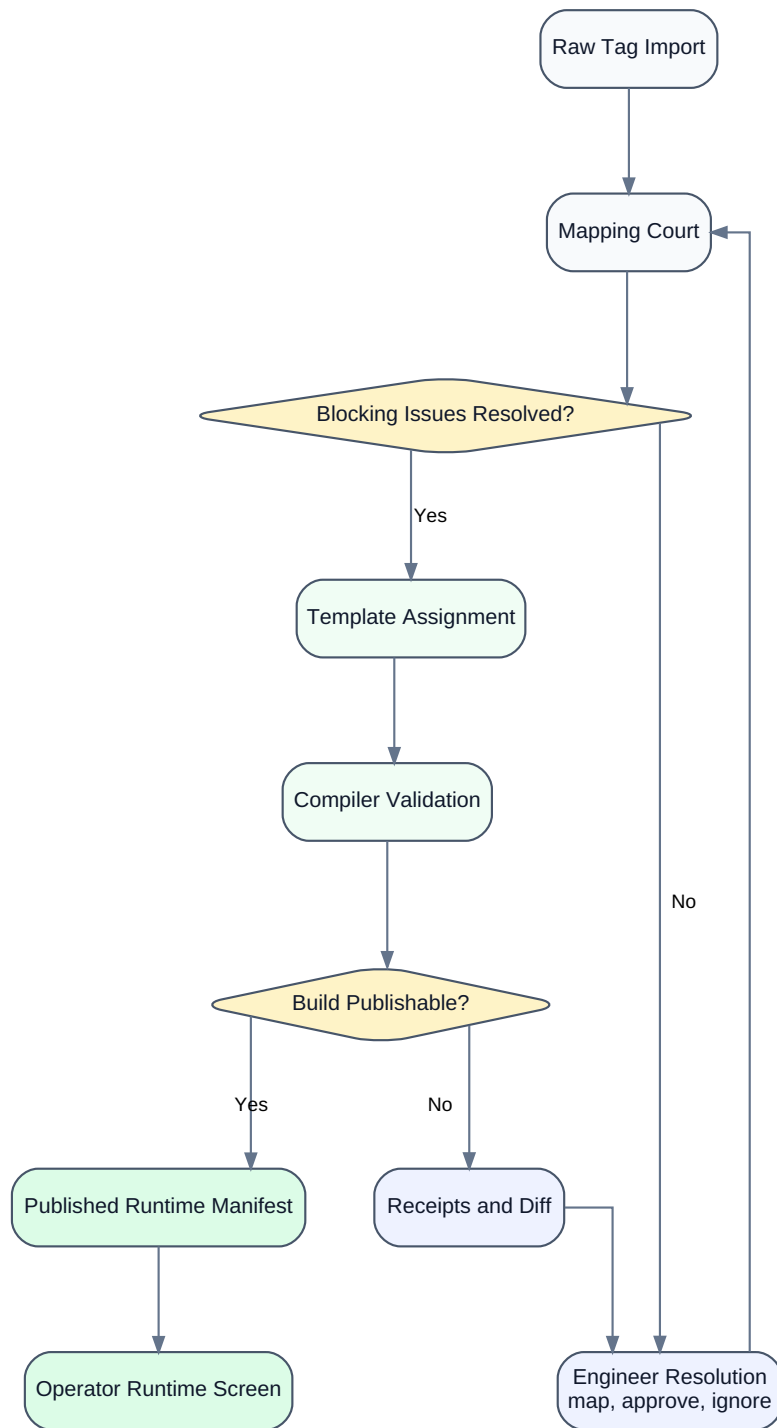


Figure 8.1 shows the Studio and HMI Compiler workflow from raw tag import to a published Runtime manifest.

9. Verification Workflow

When a value cannot be trusted from available evidence, ConfidenceOS creates a verification task. Maintenance performs the field check and captures evidence. Engineering or management can accept or reject the result. Every transition is designed to be auditable.

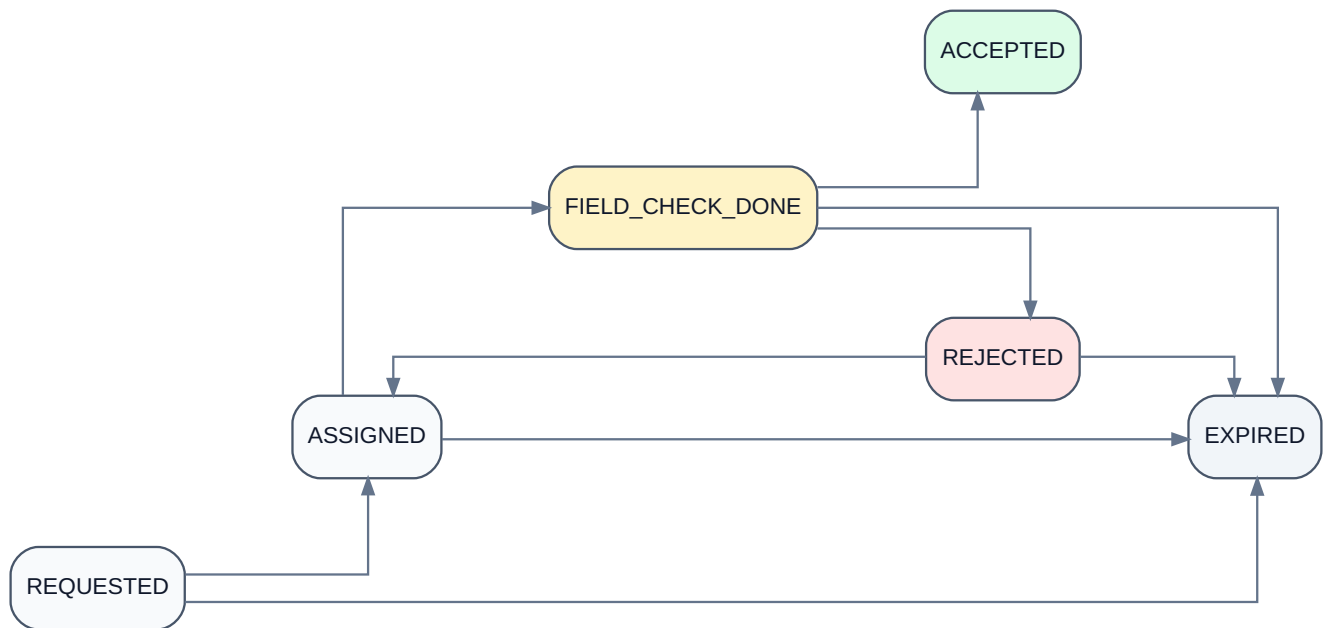


Figure 9.1 shows the field-verification lifecycle used to close the loop between operator uncertainty, maintenance action, and engineering acceptance.

10. Role-Based Product Experience

ConfidenceOS provides different experiences for different industrial users. Operators receive simplified decision guidance; maintenance receives actionable field tasks; engineers receive configuration and acceptance tools; managers and auditors receive review and traceability views.

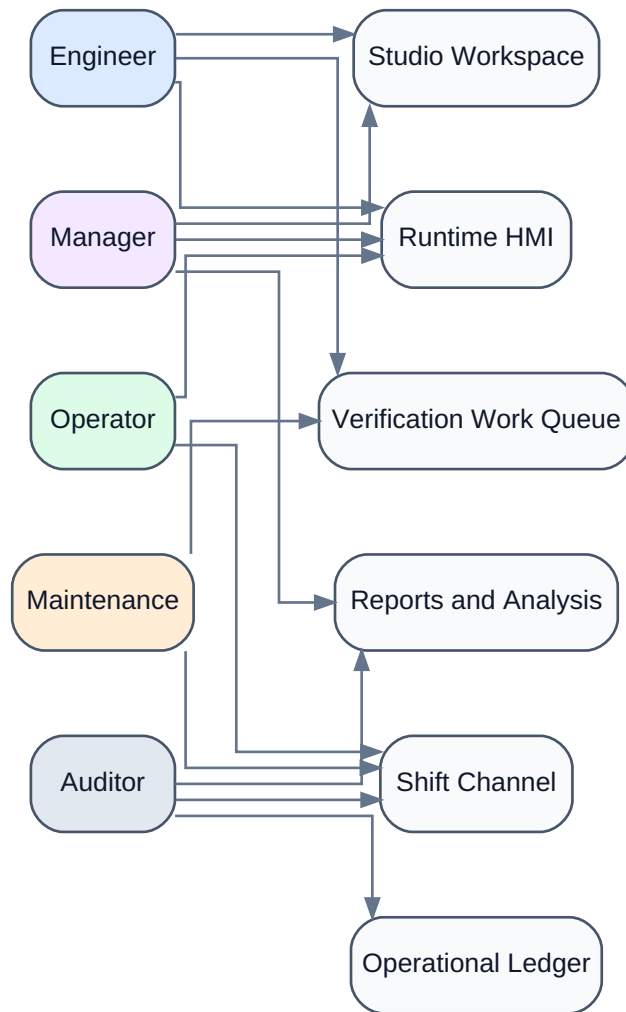


Figure 10.1 shows how user roles map to ConfidenceOS workspaces and responsibilities.

11. Core Algorithms

11.1 Confidence Scoring

The confidence engine is a governed trust rubric. It is not presented as a statistical probability. It combines calibration age, signal stability, cross-sensor consistency, and physical plausibility to create a readable trust score and evidence stack.

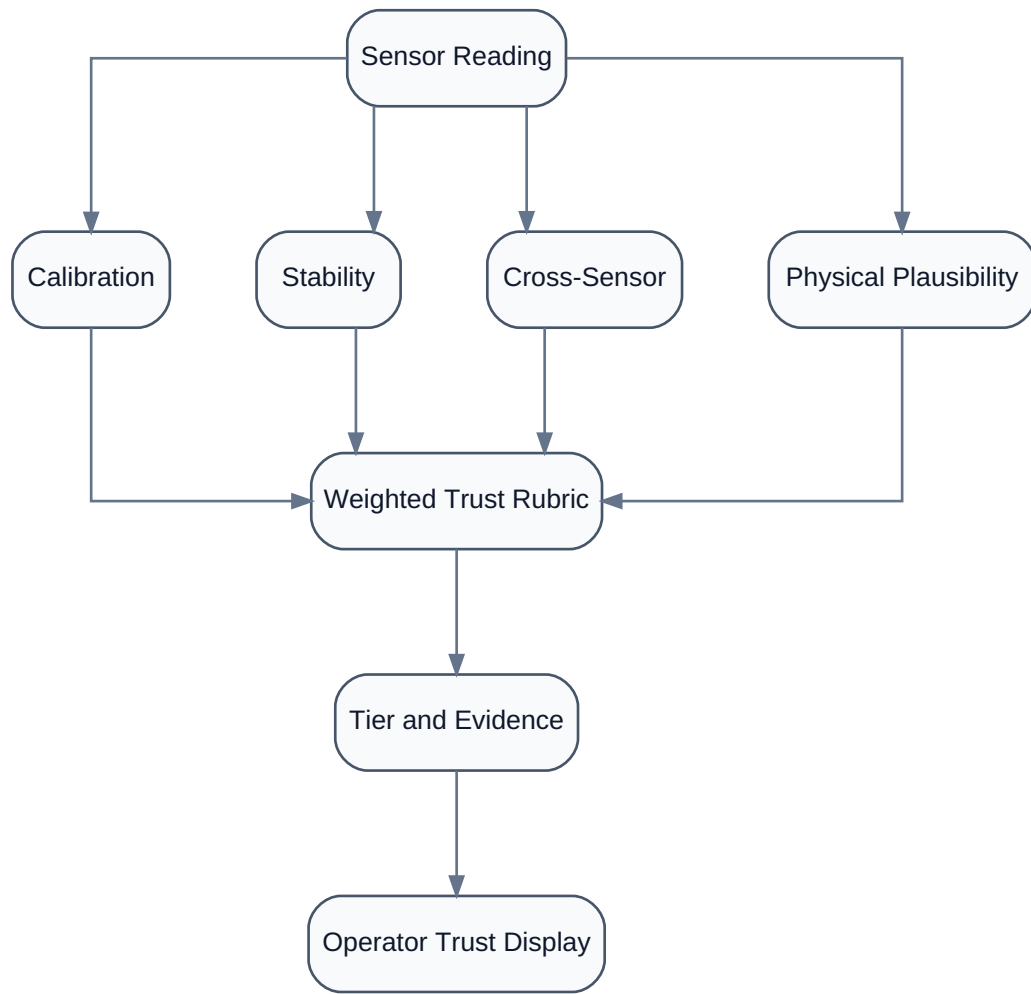


Figure 11.1 shows the confidence scoring algorithm from sensor reading to operator trust display.

```

confidence(sensor) =
  0.30 * calibration_score
+ 0.20 * stability_score
+ 0.30 * cross_sensor_score
+ 0.20 * physical_plausibility_score

```

Output tiers:

HIGH	80-100
MEDIUM	50-79
LOW	20-49
CRITICAL	0-19

11.2 Mass-Balance Residual

The mass-balance engine compares what inflow and outflow imply against what the level sensor reports. It is implemented as a single-vessel volumetric residual check, which is appropriate for demonstrating physical contradiction while keeping assumptions clear.

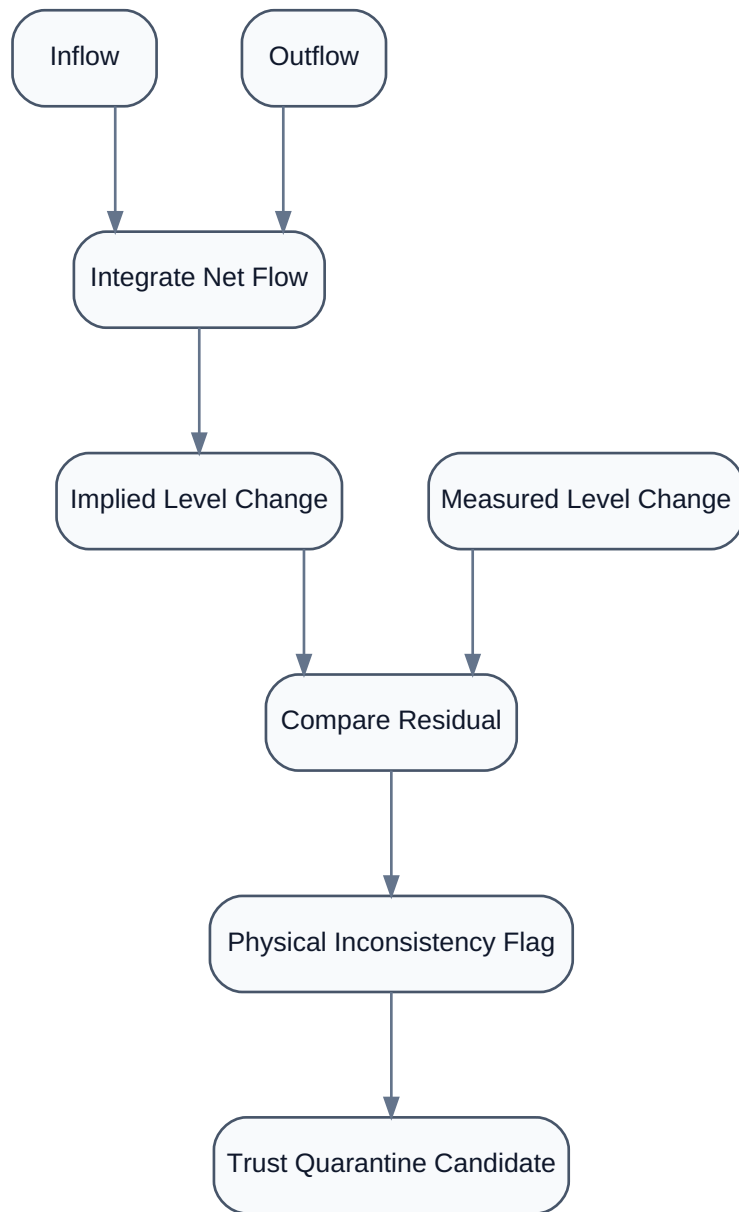


Figure 11.2 shows the mass-balance residual method used to detect physical inconsistency between flow-derived and measured level change.

```
implied_level_delta = integral(inflow_rate - outflow_rate, dt) * flow_to_level_rate
measured_level_delta = current_level - level_at_window_start
discrepancy = abs(implied_level_delta - measured_level_delta)
```

12. Functional Modules

Module	Responsibility
Sensor Simulator and Replay	Produces realistic industrial tag streams for level, flow, pressure, temperature, valve, and demo replay data.
Confidence Engine	Calculates score, tier, evidence, uncertainty, NAMUR-style condition language, and recommended action.
Mass-Balance Engine	Computes residual between flow-implied and measured level change.
Advisory Engine	Converts degraded trust and physical contradictions into abnormal situations and operator guidance.
Verification Service	Manages durable task lifecycle, procedure guidance, evidence, and audit events.
Studio Service	Maintains engineering state, Mapping Court, builds, published manifests, and build artifacts.
Shift Channel	Aggregates handover debt, notes, verification tasks, and operational ledger references.
Frontend Runtime	Presents the operator workplace, trust state, decision freeze, and single safe move.
Frontend Studio	Presents the engineering configuration and compiler workflow.
Analysis Views	Provide fleet integrity, degradation forecasts, forensics, causal graph, compliance, and engineer detail views.

13. User Interface and Workspaces

The user interface is organized around primary industrial workflows rather than generic dashboards. Runtime is the operator workplace. Studio is the engineering workspace. Work Queue is the field verification console. Shift Channel is the handover and operating-debt board.

Runtime HMI

- Live process values with trust state.
- Abnormal situation summary.
- Trust Quarantine and trusted substitutes.
- Decision freeze and single safe move.
- Mass-balance and confidence evidence.
- Read-only boundary messaging.

Studio Workspace

- Raw tag import and Mapping Court.
- Asset model and signal review.
- Template assignment and validation.
- Compiler build, receipts, publish readiness, and Runtime publication.

Work Queue

- Verification task status.
- Procedure-specific evidence requirements.

- Assignment, field check completion, acceptance, rejection, and expiry.

Shift Channel

- Handover blockers and unresolved trust debt.
- Verification tasks and evidence context.
- Operator notes and operational ledger references.

14. Technology Stack

Layer	Technology Used	Purpose
Backend API	FastAPI, Uvicorn	REST and WebSocket services for live runtime data and workflows.
Language	Python 3.11	Simulation, scoring, mass-balance, verification, and compiler logic.
Persistence	SQLite, SQLAlchemy	Demo persistence for readings, confidence logs, tasks, evidence, ledger, and build artifacts.
Frontend	React 19, Vite 8	Role-aware Runtime, Studio, Work Queue, Shift Channel, and analysis views.
State and Charts	Zustand, Recharts	Live WebSocket state and visualization of trends and process evidence.
AI Assistance	Anthropic SDK with fallback behavior	Advisory explanations, handover text, and mapping support when configured.
Styling	Tailwind CSS 4, custom design tokens	Industrial HMI-style UI with disciplined status presentation.
Deployment	Docker-ready backend and Vite build	Demo-friendly deployment with API and WebSocket routing.

15. Database Design

The persistence layer stores readings, confidence history, alarms, verification tasks, evidence, audit events, operational ledger events, Studio state, build artifacts, and users. This allows the system to support both live display and traceability.

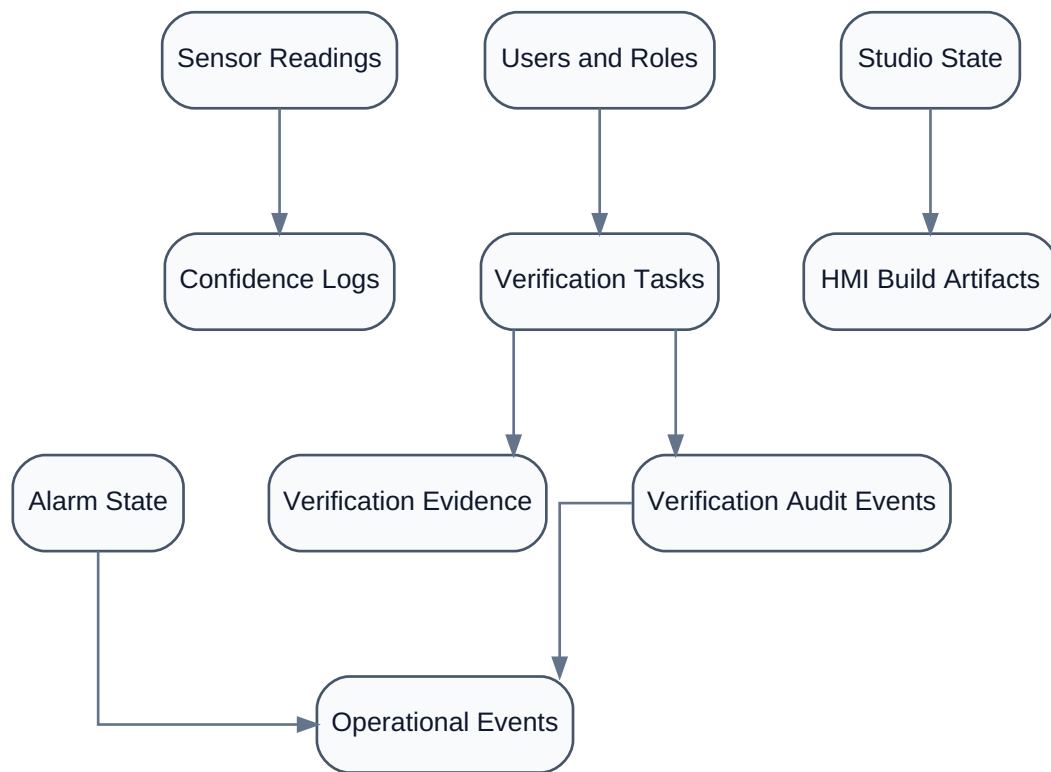


Figure 15.1 shows the main persistence entities used by ConfidenceOS.

16. API and Integration Overview

The backend exposes REST endpoints for runtime state, confidence, mass balance, generated screens, Studio, verification, handover, fleet integrity, predictions, and operational ledger. The live Runtime data stream is provided through WebSocket.

Area	Representative Interfaces
Live Runtime	/ws/sensors, /api/sensors/latest, /api/confidence, /api/mass-balance/state
Generated Screens	/api/screens/generated, /api/runtime/navigation, /api/runtime/situations
Studio	/api/studio, /api/studio/mapping-court, /api/studio/build/run, /api/studio/publish
Verification	/api/verification-tasks, /api/verification-tasks/audit, /api/verification-tokens
Handover	/api/shift-channel, /api/shift-channel/note, /api/handover/debt
Analysis	/api/fleet, /api/fleet/history, /api/predictions, /api/operational-ledger

17. ABB Challenge Alignment

Evaluation Area	ConfidenceOS Response
Problem clarity	Targets false certainty in HMI readings rather than only alarm count.
Innovation	Adds explicit trust state, Trust Quarantine, substitute evidence, and handover debt to HMI operation.
Industrial relevance	Uses instrumentation concepts such as calibration, stability, cross-sensor consistency, mass balance, field verification, and operator handover.
Model-based engineering	Studio turns asset models, tags, templates, mappings, validation, and receipts into generated Runtime screens.
Role awareness	Operators see decision guidance, maintenance sees field tasks, engineers see mappings and builds, auditors see traceability.
Safety posture	System remains read-only and does not replace DCS, SIS, or operator authority.

18. Safety and Read-Only Boundary

ConfidenceOS is deliberately designed as a read-only layer. It does not send control commands, change setpoints, operate valves or pumps, acknowledge plant alarms in the control system, or replace safety systems.

ConfidenceOS reads process data.
ConfidenceOS computes trust, evidence, and operating guidance.
ConfidenceOS displays decision-support information.
ConfidenceOS does not control the plant.

This boundary is important because the product is meant to increase operator awareness without creating a new unsafe control path.

19. Testing and Validation Approach

The validation approach focuses on proving that the system behaves coherently across roles and workflows.

- Verify login and role-based navigation for all roles.
- Verify Runtime loads and live WebSocket updates are visible.
- Verify confidence score changes under simulated failure modes.
- Verify mass-balance divergence creates visible operator evidence.
- Verify Trust Quarantine changes operating-basis guidance.
- Verify verification tasks move through legal lifecycle states.
- Verify evidence is required before acceptance.
- Verify Studio blocks publish when mappings are unresolved.
- Verify Shift Channel preserves unresolved operating debt.
- Verify ledger and audit views expose traceable events.

20. Current Boundaries and Future Scope

The current implementation is a strong project prototype. It is intentionally honest about what is simulated, what is heuristic, and what would require additional production engineering.

Current Boundary	Future Enhancement
SQLite is used for demo persistence.	Move to PostgreSQL or an industrial time-series/historian-backed architecture.
OPC UA is prepared as a read-only boundary, not production-connected by default.	Add secure subscriptions, certificates, namespace mapping, quality flags, and reconnect handling.
Mass balance is single-vessel and volumetric.	Support multi-unit, multi-stream, density-aware, delay-aware process models.
Confidence score is a governed trust rubric.	Calibrate against plant history, maintenance records, and validated instrumentation data.
AI is advisory.	Keep AI bounded, explainable, and subordinate to deterministic validation.
JWT auth is suitable for demonstration.	Integrate enterprise identity, SSO, RBAC, and signed audit trails.

21. Expected Impact

For Operators

Faster recognition of what is wrong, what not to trust, and what action is safe.

For Maintenance

Clearer field tasks, procedure-specific evidence, and closure criteria.

For Engineers

Repeatable tag mapping, template assignment, validation, build receipts, and published Runtime screens.

For Managers

Better visibility into unresolved operating debt and handover readiness.

For Auditors

Traceable verification, handover, and operational events.

For Industrial Software Evaluation

A clear next-generation HMI concept that combines operator UX, engineering workflow, and trust-aware runtime intelligence.

22. Conclusion

ConfidenceOS reframes the purpose of an industrial HMI. A traditional HMI shows values. ConfidenceOS shows values, trust, evidence, consequences, verification needs, and handover obligations.

The project combines confidence scoring, physical residual checking, Trust Quarantine, generated Runtime screens, role-aware workflows, structured verification, Shift Channel continuity, and operational auditability. It remains read-only and respects the boundary between advisory intelligence and process control.

ConfidenceOS is the missing trust layer beside the control system.

ConfidenceOS Field Mobile — Complete Technical Documentation

- **Version:** 1.0.0
 - **Deployment:** Progressive Web App (PWA)
 - **Live URL:** Netlify (deployable from `dist/` folder)
 - **Local Dev Port:** 5175
 - **Backend API:** `https://cfosbd.karthiksurkanti.in`
-

1. Overview

ConfidenceOS Field Mobile is a standalone Progressive Web App (PWA) designed for plant floor personnel — operators, maintenance technicians, engineers, and managers — who need real-time plant operations data on their mobile phones.

The app is a **read-only advisory system**. It does not write control commands or operate field devices. It surfaces:

- Live instrument alarms with ISA-18.2 priority classification
- Active plant incidents and shift handover debt
- Field verification tasks assigned by the confidence system
- Shift handover briefings including equipment status and open items
- Push notifications for critical alarms delivered to the device

The app works in two modes:

Mode	How it activates	Data source
Demo	Demo credentials (role buttons on login)	Mock static data
Live	Real backend credentials	Backend API + WebSocket

In Demo mode, a background JWT upgrade attempt runs silently — if the backend is reachable, the app seamlessly transitions to Live mode without any visible disruption to the user.

2. Technology Stack

Layer	Technology	Version
Build tool	Vite	^6.0.0
UI framework	React	^19.0.0

Layer	Technology	Version
Routing	React Router DOM	^7.0.0
State management	Zustand (with persist middleware)	^5.0.0
PWA support	vite-plugin-pwa	^0.21.0
Styling	Custom mobile-first CSS (no framework)	—
Push notifications	Web Push API (VAPID)	—
Real-time updates	WebSocket	—

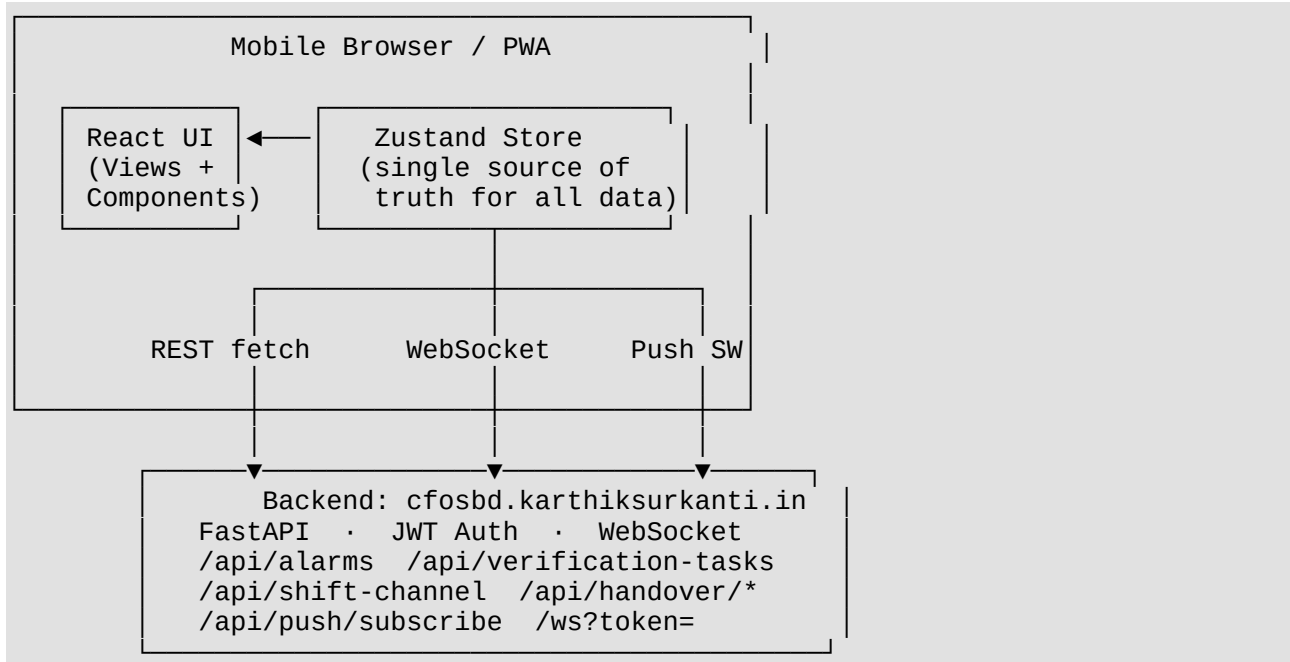
No UI component library (MUI, Tailwind, etc.) is used. All styles are hand-crafted in `src/index.css` using CSS custom properties (design tokens), keeping the bundle lean and load times fast on mobile networks.

3. Project Directory Structure

D:\ABB\mobile\	
— index.html	← App entry HTML (mounts React root)
— package.json	← Dependencies and scripts
— vite.config.js	← Vite + PWA plugin configuration
— public/	
— manifest.json	← PWA manifest (name, icons, display mode)
— favicon.svg	← App icon (SVG shield)
— icon-192.png	← PWA home screen icon (192×192)
— _redirects	← Netlify SPA redirect rule (<code>/* → /index.html</code>)
— src/	
— main.jsx	← React root mount + BrowserRouter wrapper
— App.jsx	← Auth gate, routing, post-login logic
— store.js	← Zustand global state (auth + data + API calls)
— index.css	← All styles: design tokens, layout, components
— mock/	
— data.js	← Static demo fixtures (alarms, incidents,
tasks...)	
— views/	
— LoginView.jsx	← Full-screen login with role quick-sign-in
— AlarmsView.jsx	← Alarm feed with swipe-to-acknowledge
— IncidentsView.jsx	← Active incidents + handover debt
— TasksView.jsx	← Verification task queue with filters
— HandoverView.jsx	← Shift handover brief and equipment status
— SettingsView.jsx	← Account info, notifications toggle, dark mode
— components/	
— BottomNav.jsx	← Role-filtered tab bar (fixed at bottom)
— AlarmCard.jsx	← Alarm row with swipe gesture + live timer
— AlarmDetailSheet.jsx	← Bottom-sheet modal with full alarm details
— IncidentCard.jsx	← Incident summary card with open duration
— TaskCard.jsx	← Task row with checkbox completion
— PriorityBadge.jsx	← P1/P2/P3 colored pill component

4. Architecture

4.1 High-Level Architecture



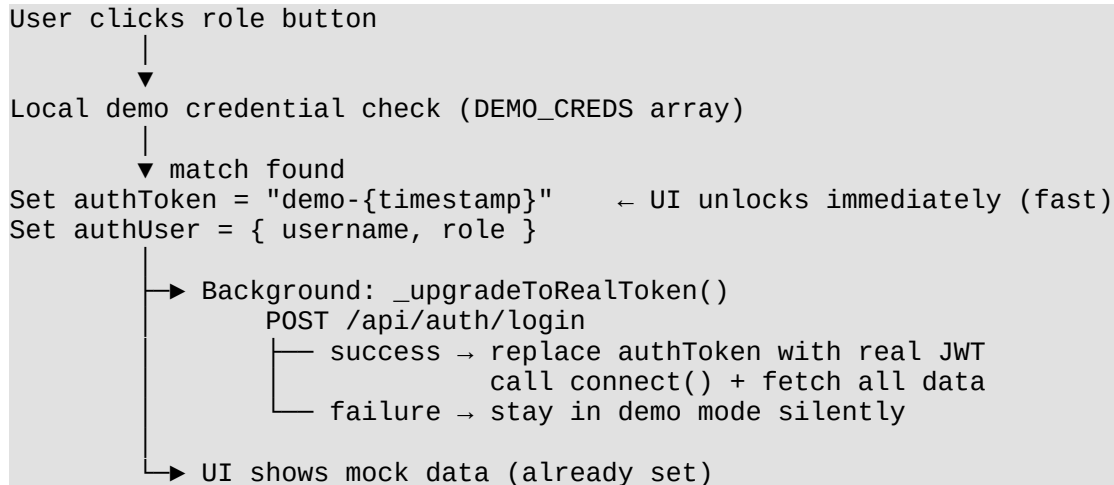
4.2 State Architecture (Zustand Store)

The entire application state lives in one Zustand store (`src/store.js`). It is divided into logical slices:

```
store
├── Auth slice
│   ├── authToken      ← JWT or "demo-{timestamp}" string
│   ├── authUser       ← { username, role }
│   ├── authLoading    ← spinner state during login
│   └── authError      ← error message for login failures
├── Preferences slice (persisted to localStorage)
│   ├── darkMode       ← boolean
│   └── notificationsEnabled ← boolean
├── Live state slice
│   └── connected       ← WebSocket connection status
└── Data slice
    ├── alarms          ← array of alarm objects
    ├── incidents        ← array of incident objects
    ├── handoverDebt     ← array of unresolved debt entries
    ├── tasks           ← array of verification task objects
    └── handoverData     ← full shift handover object
```

Persistence: Only `authToken`, `authUser`, `darkMode`, and `notificationsEnabled` are persisted to `localStorage` (key: `cfos-field-v1`). Live data is always re-fetched on page load — it is never stale-cached.

4.3 Demo / Live Token Pattern



This pattern ensures the app is instant — the user is never blocked waiting for the backend.

5. Authentication Flow

5.1 Login

1. User opens the app → sees `LoginView` (auth gate in `App.jsx` blocks all routes when `authToken === null`)
2. User taps a role button or enters credentials manually
3. `store.login(username, password)` is called:
 - Checks against `DEMO_CREDS` first (works offline)
 - If no match, calls `POST /api/auth/login` with form-encoded body
4. On success: `authToken` and `authUser` are set in the store
5. `App.jsx` detects `authUser` change via `useEffect` → navigates to role's home screen
6. 1.5 seconds after login: notification permission prompt fires automatically (if not already decided)

5.2 JWT Authentication

All backend requests include the JWT as a Bearer token:

```
Authorization: Bearer <jwt>
```

The `isRealToken()` helper in the store returns `false` for demo tokens (demo - * prefix), preventing any API call from firing in demo mode.

5.3 Post-Login Data Fetch

Immediately after a real token is available (either real login or JWT upgrade), these fire in parallel:

- `connect()` — opens WebSocket

- `fetchAlarms()` — loads alarm list
- `fetchTasks()` — loads verification task queue
- `fetchIncidents()` — loads active incidents from shift channel
- `fetchHandover()` — loads handover debt + latest brief

5.4 Session Persistence

`authToken` and `authUser` survive page refresh (stored in `localStorage`). On page load, `App.jsx` detects a persisted real token and automatically reconnects the `WebSocket` + re-fetches all data.

5.5 Logout

`store.logout()` closes the `WebSocket`, clears all auth state, and resets all data back to mock values.

6. Role-Based Access Control

Five roles are supported. Each role has a defined home screen and set of visible tabs.

Role	Home Screen	Alarms	Incidents	Tasks	Handover	Settings
Operator	Alarms	✓	✓	✓	✓	✓
Maintenance	Tasks	✓	✓	✓	✓	✓
Engineer	Tasks	✓	✓	✓	✓	✓
Manager	Incidents	✓	✓	✓	✓	✓
Auditor	Incidents	✓	✓	—	✓	✓

Role filtering is enforced in `BottomNav.jsx` — tabs are rendered only if the current role is in each tab's `roles` array. The routes themselves are not guarded (redirecting `/tasks` for an Auditor would just show the Tasks view with no harm — the role context is displayed only).

Demo Credentials

Role	Username	Password
Operator	operator	Confidence0S-Op-2025
Maintenance	maint	Confidence0S-Maint-2025
Engineer	engineer	Confidence0S-Eng-2025
Manager	manager	Confidence0S-Mgr-2025
Auditor	auditor	Confidence0S-Aud-2025

7. Screen-by-Screen Walkthrough

7.1 Login Screen (`LoginView.jsx`)

Purpose: Entry point for all users. Provides both quick role-based sign-in and a manual username/password form.

Layout:

- Left panel (desktop) / top strip (mobile): ConfidenceOS branding, feature list, read-only disclaimer
- Right panel: Role quick-sign-in buttons + manual form with OR divider

Behavior:

- Clicking a role button auto-submits with demo credentials — user is signed in within ~350ms
- Role buttons show a spinner on the active button while loading
- If the backend is unreachable, demo mode still works (offline-capable login)
- Error messages appear in a red banner below the form

7.2 Alarms Screen (`AlarmsView.jsx`)

Default home for: Operator

Purpose: Real-time alarm triage sorted by ISA-18.2 priority (P1 → P2 → P3). Acknowledged alarms sink to the bottom.

Key interactions:

- **Swipe left** on any alarm card → reveals red background → release past 90px threshold to acknowledge
- **Tap** an alarm card → opens `AlarmDetailSheet` bottom sheet with full alarm details
- **Acknowledge button** on the card (secondary option to swipe)
- **Ack All** button in the summary strip when 2+ unacked alarms exist

Summary strip: Shows count of unacknowledged alarms and count of P1 critical alarms. Turns red when alarms exist, green when all clear.

Live updates: `AlarmCard` has an internal 30-second timer that re-renders the "X minutes ago" timestamp without refetching.

Backend wiring: `fetchAlarms()` → GET `/api/alarms`. `acknowledgeAlarm(id)` → optimistic local update first, then POST `/api/alarms/acknowledge`.

Data shape:


```
{
  id:      string,      // alarm identifier
  tag:      string,      // sensor tag (e.g. "PT-101")
  desc:      string,      // human-readable description
  priority: "P1"|"P2"|"P3",
  area:      string,      // plant area (e.g. "Reactor")
  raisedAt: number,      // Unix ms timestamp
  acked:     boolean
}
```

7.3 Incidents Screen (IncidentsView.jsx)

Default home for: Manager, Auditor

Purpose: Shows all active plant incidents and unresolved handover debt carried over from the previous shift.

Sections:

1. **Active Incidents** — each card shows title, plant area, severity badge (High/Medium/Low), and how long the incident has been open (live-computed from `openSince` timestamp)
2. **Handover Debt** — entries flagged as unresolved from the prior shift, tagged with urgency level and originating shift name

Backend wiring:

- `fetchIncidents()` → GET `/api/shift-channel` (extracts `.incidents` array)
 - WebSocket also pushes incident updates in real-time via `msg.incidents` field
 - `fetchHandover()` → GET `/api/handover/debt` for handover debt entries
-

7.4 Tasks Screen (TasksView.jsx)

Default home for: Maintenance, Engineer

Purpose: Field verification task queue — lists all open and completed instrument verification tasks generated by the ConfidenceOS confidence system.

Filter bar: Pill buttons — All / Pending / In Progress / Done

Task card shows:

- Sensor tag (e.g. LT-5100)
- Verification description (e.g. "local_field_check")
- Assigned role badge
- Status label (color-coded)
- Due-in countdown (computed from task expiry time)

Completing a task: Tapping the checkbox sets the task to DONE locally (instant UI) and fires

POST /api/verification-tasks/state with {state: "ACCEPTED"} in the background.

Backend wiring: fetchTasks() → GET /api/verification-tasks. Tasks originate from the ConfidenceOS trust quarantine system — they are created automatically when sensor confidence scores drop below acceptable thresholds.

State mapping:

Backend state	Mobile status
REQUESTED	PENDING
IN_PROGRESS	IN_PROGRESS
ACCEPTED	DONE
EXPIRED	DONE

7.5 Handover Screen (HandoverView.jsx)

Purpose: Full shift handover brief — allows incoming shift to understand exactly what happened, what equipment needs watching, and what was left unresolved.

Sections:

1. **Shift meta bar** — shift name, supervisor name, shift start time
2. **Open Items** — urgency-tagged list of items requiring attention (High/Medium/Low)
3. **Equipment Status** — sensor tag rows with status pip (green/amber/red) and area label
4. **Notes from Last Shift** — free-text narrative from outgoing shift

Backend wiring: fetchHandover() fires two requests in parallel:

- GET /api/handover/latest → populates shift meta, open items, equipment status, notes
- GET /api/handover/debt → populates the handover debt shown in Incidents view

If the backend has no generated brief yet (brief is null), mock data is retained as fallback.

Record Handover button: Currently shows a "coming soon" toast. Future: will submit a new handover entry.

7.6 Settings Screen (SettingsView.jsx)

Purpose: Account information, preference toggles, and system diagnostics.

Sections:

1. **Account** — avatar with role initial, username, role badge

2. **Current Status** — live counts of unacknowledged alarms and pending tasks (pulled from store)
3. **Preferences** — Dark Mode toggle (persisted), Push Notifications toggle
4. **About** — Live/Demo mode indicator, version number, read-only disclaimer

Push notification toggle: Calls `enableNotifications()` which:

1. Calls `Notification.requestPermission()` → browser native OS-level prompt
2. Gets the service worker push subscription
3. Posts the subscription to `POST /api/push/subscribe` with the JWT

8. Real-Time Updates (WebSocket)

The app maintains a persistent WebSocket connection for live data delta updates.

Connection: `wss://cfosbd.karthiksurkanti.in/ws?token=<jwt>`

Reconnect strategy: On disconnect, waits 5 seconds then reconnects automatically, as long as a real JWT is still in the store.

Message format (server → client):

```
{
  "unacked_alarms": 3,
  "incidents": [...],
  "handover_debt": {
    "entries": [...]
  }
}
```

Handling logic:

- `unacked_alarms`: If count differs from current local state → triggers `fetchAlarms()` for a fresh pull
- `incidents`: If present → directly replaces `store.incidents`
- `handover_debt.entries`: If present → directly replaces `store.handoverDebt`

The WebSocket only opens with a real JWT. Demo-mode users have no WebSocket and only see the static mock data.

9. Push Notifications

The app implements the Web Push API with VAPID key authentication.

Flow

User toggles notifications ON



```

Notification.requestPermission() ← native browser/OS dialog
    ↓
    ▼ "granted"
navigator.serviceWorker.ready
    ↓
    ▼
reg.pushManager.subscribe({
  userVisibleOnly: true,
  applicationServerKey: VAPID_PUBLIC_KEY
})
    ↓
    ▼
POST /api/push/subscribe
  { endpoint, keys: { p256dh, auth } }
    ↓
    ▼
Server registers subscription → can now push alarms to device

```

VAPID Public Key:

B04lfShf4rqt3QxHaAdoey01DwccHqHzXptI9LGH4Iqm0ddgaTZ1cHa9wR6fM7dNyI
v9lzjMs0QFXIURTywrrM0

Auto-prompt on first login: When a user logs in for the first time and `Notification.permission === 'default'` (permission not yet decided), the app automatically triggers the permission request after a 1.5-second delay — matching the pattern used by Android apps.

Important behavior: Permission state is shared between the browser tab and the installed PWA. If permission is granted in Chrome, the installed PWA inherits that state and will not show the prompt again (correct behavior — it is already enabled).

10. PWA (Progressive Web App) Configuration

Install Behavior

- Users see an "Add to Home Screen" / "Install" banner (PWA install prompt captured in `App.jsx`)
- The app runs in **standalone** mode — no browser chrome, full-screen like a native app
- Orientation locked to portrait

Service Worker Strategy

Configured as `injectManifest` in `vite.config.js` — a custom `src/sw.js` service worker is used rather than the auto-generated one. This allows control over caching strategy and push notification handling.

Manifest

```

{
  "name": "ConfidenceOS Field",
  "short_name": "ConfidenceOS",
  "display": "standalone",

```

```
"theme_color": "#005aa0",
"background_color": "#bfbfbf",
"start_url": "/"
}
```

Netlify Deployment

The `public/_redirects` file contains:

```
/* /index.html 200
```

This ensures React Router works correctly — all URL paths return `index.html` and the client-side router handles navigation.

11. Design System

Color Tokens (CSS Custom Properties)

Token	Light value	Purpose
--bg-base	#bfbfbf	Page background
--bg-card	#d7d7d7	Card surface
--bg-elevated	#e6e6e6	Inputs, elevated surfaces
--border	#8f8f8f	Card borders
--text	#111111	Primary text
--text-muted	#3a3a3a	Secondary text
--text-dim	#5a5a5a	Tertiary / metadata text
--primary	#005aa0	Brand blue, buttons
--safe	#2f6b2f	Safe/OK status
--warning	#9b6200	Warning status
--critical	#b00020	Critical status
--alarm-p1	#cc0000	P1 alarm (red)
--alarm-p2	#ffcc00	P2 alarm (yellow)
--alarm-p3	#ff8c33	P3 alarm (orange)

Dark mode inverts to a dark slate palette, toggled by `data-theme="dark"` on `<html>`.

Role Colors

Role	Color
Operator	#22c55e (green)
Maintenance	#f97316 (orange)
Engineer	#0a84ff (blue)

Role	Color
Manager	#a855f7 (purple)
Auditor	#9ca3af (grey)

Layout

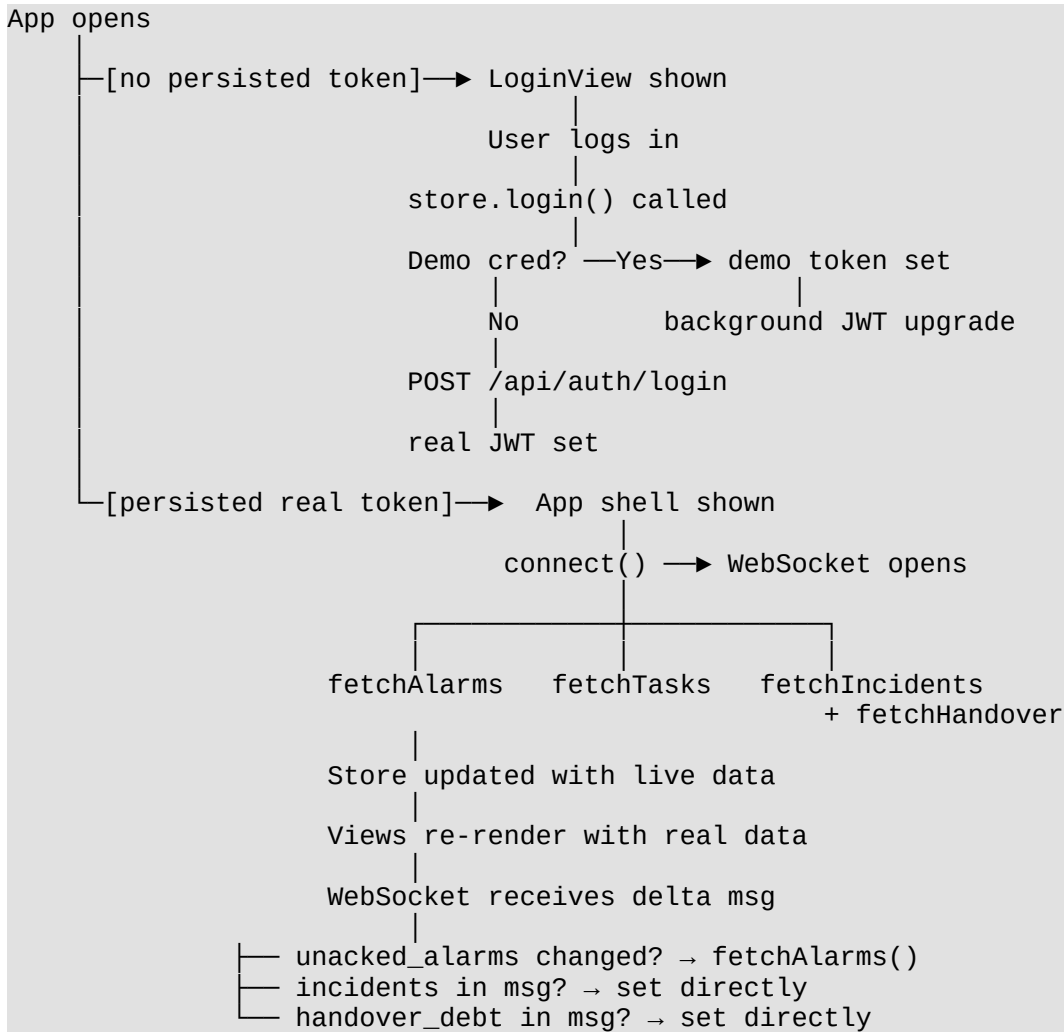
- **Mobile app shell:** `display: flex; flex-direction: column; height: 100dvh`
- **Header:** Fixed 52px top bar — brand, live connection pip, role badge, dark mode toggle
- **Content area:** `flex: 1; overflow-y: auto; padding-bottom: 72px` (clearance for bottom nav)
- **Bottom nav:** Fixed 64px bar — equal-width tab buttons with icon + label

12. Backend API Reference

All endpoints are on <https://cfosbd.karthiksurkanti.in>.

Method	Endpoint	Purpose
POST	/api/auth/login	Login — form-encoded username + password, returns <code>access_token</code>
GET	/api/alarms	List all alarms for the plant
POST	/api/alarms/acknowledge	Acknowledge an alarm by <code>alarm_id</code>
GET	/api/verification-tasks	List all field verification tasks
POST	/api/verification-tasks/state	Update a task state (ACCEPTED, etc.)
GET	/api/shift-channel	Shift channel data including active incidents
GET	/api/handover/latest	Latest generated handover brief
GET	/api/handover/debt	Unresolved handover debt entries
POST	/api/push/subscribe	Register a Web Push subscription for the user
WS	/ws?token=<jwt>	Real-time delta updates stream

13. Data Flow — Complete Lifecycle



14. Build and Deployment

Development

```
cd D:\ABB\mobile
npm install
npm run dev
```

```
# → http://localhost:5175
```

Production Build

```
npm run build
```

```
# → generates D:\ABB\mobile\dist\
```

Deploy to Netlify

1. Run `npm run build`
2. Drag and drop the `dist/` folder into Netlify's deploy drop zone

3. No build command configuration needed — the `dist/` folder is the final artifact

File Structure of `dist/` (after build)

```
dist/
├── index.html
├── _redirects          ← copied from public/
├── manifest.json      ← copied from public/
├── favicon.svg
├── icon-192.png
├── sw.js              ← compiled service worker
├── assets/
│   ├── index-[hash].js
│   └── index-[hash].css
```

15. What's Currently Mock vs Live

Feature	Status	Notes
Alarms	Live	Fetched from <code>/api/alarms</code> + WebSocket delta
Alarm acknowledgement	Live	POST <code>/api/alarms/acknowledge</code>
Incidents	Live	Fetched from <code>/api/shift-channel</code> + WebSocket delta
Verification tasks	Live	Fetched from <code>/api/verification-tasks</code>
Task completion	Live	POST <code>/api/verification-tasks/state</code>
Handover debt	Live	Fetched from <code>/api/handover/debt</code>
Handover brief	Live	Fetched from <code>/api/handover/latest</code> (falls back to mock if no brief generated yet)
Push notifications	Live	Full VAPID Web Push subscribe flow
Record Handover	Stub	Button shows toast — submission endpoint not yet implemented
Demo mode fallback	Always	All screens show meaningful mock data when backend is unreachable

16. Known Constraints and Security Notes

- **Read-only advisory system** — the app has no ability to send control commands to the plant. It only reads alarm, incident, task, and handover data.
- **JWT expiry** — the app does not automatically refresh expired JWTs. If the token expires, the user must log in again.
- **No biometric/PIN lock** — the app relies solely on the backend JWT. If the device is unlocked and shared, any user can access the session until logout.
- **Offline support** — demo mode works fully offline. Live mode requires connectivity; the service worker caches app shell assets so the UI loads without network, but live data requires the backend.
- **Notification permission on installed PWA** — permission state is shared between the browser and the installed PWA. Once permission is granted or denied in the browser, the installed app inherits that state. Resetting requires the user to visit Chrome → Site Settings → Notifications → Reset for the app's URL.